

MEMORIA TECNICA Y FUNCIONAL

LocalLift Studio

Documentacion profesional tipo TFG sobre una plataforma de digitalizacion integral para negocios locales: web, CRM, reservas, tienda, chatbot, analitica, despliegue y roadmap.

FECHA**26/05/2026****ESTADO****MVP
avanzado****ENTREGA****Demo
vendible****REPO****GitHub**

INDICE

LocalLift Studio

Memoria tecnica, funcional y comercial del proyecto

0. Resumen ejecutivo

1. Introduccion

1.1 Contexto del problema

1.2 Motivacion

1.3 Enfoque

2. Objetivos del proyecto

2.1 Objetivo general

2.2 Objetivos especificos

2.3 Criterios de exito

3. Alcance actual

3.1 Lo que incluye el MVP avanzado

3.2 Lo que no incluye todavia

4. Vision de producto

4.1 Producto final deseado

4.2 Principios de diseno del producto

4.3 Propuesta de valor

5. Analisis competitivo

5.1 Wix, Squarespace, Webflow y Framer

5.2 Shopify y Square

5.3 Odoo y Zoho

5.4 Chatbots y helpdesks

6. Arquitectura general

6.1 Vista de alto nivel

6.2 Frontend

6.3 Backend

6.4 Persistencia

6.5 Despliegue

7. Estructura de carpetas

7.1 Archivos principales

8. Funcionalidades del Studio

8.1 Editor de datos del negocio

8.2 Vista previa responsive

8.3 Temas visuales

8.4 Personalizacion avanzada

8.5 Demo en vivo

8.6 Checklist de calidad

8.7 Exportacion HTML

9. Web publica generada

9.1 Secciones principales

9.2 Conversion

9.3 Fallback standalone

9.4 Analitica de eventos

10. Chatbot

10.1 Chatbot local

10.2 Configuracion

10.3 Captura de leads

10.4 Endpoint IA opcional

10.5 Limitaciones actuales

11. CRM y leads

11.1 Objetivo

11.2 Modelo de contacto

11.3 Estados

11.4 Actividades

11.5 Endpoints

11.6 Portal

11.7 CSV

12. Reservas y agenda

12.1 Objetivo

12.2 Servicios reservables

12.3 Disponibilidad semanal

12.4 Bloqueos manuales

12.5 Validacion de reservas

12.6 Recordatorios

- 12.7 Vista calendario
- 12.8 Exportacion CSV
- 13. Tienda, pagos y pedidos
 - 13.1 Tienda visual en la web
 - 13.2 Backend de comercio
 - 13.3 Seguridad del backend de comercio
 - 13.4 Limitaciones actuales
- 14. Reportes y analitica
 - 14.1 Objetivo
 - 14.2 API de reporte
 - 14.3 Vista imprimible
 - 14.4 Recomendaciones automaticas
 - 14.5 Importancia comercial
- 15. Google Ops y SEO local
 - 15.1 Campos Google-ready
 - 15.2 Backend Google de ejemplo
 - 15.3 Playbook Google
 - 15.4 Pendiente
- 16. Seguridad
 - 16.1 Estado actual
 - 16.2 Validacion de entorno
 - 16.3 Limitaciones
- 17. Despliegue
 - 17.1 Local
 - 17.2 Produccion
 - 17.3 Render
 - 17.4 Docker
 - 17.5 Frontend separado
- 18. Guion de defensa tecnica tipo TFG
 - 18.1 Apertura
 - 18.2 Problema
 - 18.3 Solucion
 - 18.4 Demo tecnica
 - 18.5 Decisiones de arquitectura

- 18.6 Seguridad
- 18.7 Resultados
- 18.8 Cierre
- 19. Guion comercial de demo
 - 19.1 Entrada
 - 19.2 Demo movil primero
 - 19.3 Mostrar operacion
 - 19.4 Cierre
 - 19.5 Objeciones
- 20. Guion para inversores
 - 20.1 Tesis
 - 20.2 Mercado
 - 20.3 Producto
 - 20.4 Modelo de negocio
 - 20.5 Traccion tecnica
 - 20.6 Siguiente hito
- 21. Modelo comercial
 - 21.1 Paquete Presencia Local
 - 21.2 Paquete Operacion Local
 - 21.3 Paquete Comercio Local
 - 21.4 Paquete Digitalizacion Total
- 22. Roadmap pendiente
 - 22.1 Prioridad inmediata
 - 22.2 Prioridad tecnica alta
 - 22.3 Prioridad de producto
 - 22.4 Integraciones
- 23. Riesgos
 - 23.1 Riesgo tecnico: JSON en produccion
 - 23.2 Riesgo comercial: vender demasiado pronto
 - 23.3 Riesgo de soporte
 - 23.4 Riesgo de integraciones
 - 23.5 Riesgo de seguridad
- 24. Plan de validacion
 - 24.1 Pruebas tecnicas

24.2 Pruebas UX

24.3 Pruebas comerciales

25. Manual de uso rapido

25.1 Crear web

25.2 Gestionar negocio

25.3 Desplegar

26. Estado actual por porcentaje

26.1 Demo vendible online

26.2 Producto completo Plan Maestro

27. Conclusiones

Anexo A. Endpoints principales

Anexo B. Variables de entorno

Anexo C. Checklist de entrega a cliente

Anexo D. Glosario

LocalLift Studio

Memoria tecnica, funcional y comercial del proyecto

Documento tipo TFG / memoria profesional Estado: version consolidada para demo, piloto y evolucion a producto **Fecha:** 26 de mayo de 2026 **Repositorio:**

<https://github.com/luicorcob/cascos> **Producto:** plataforma de digitalizacion integral para negocios locales

0. Resumen ejecutivo

LocalLift Studio es una plataforma orientada a digitalizar negocios locales de forma rapida, profesional y operativa. El proyecto parte de una idea sencilla: muchos pequenos negocios necesitan una presencia digital atractiva, pero no quieren aprender herramientas complejas, contratar varios proveedores ni mantener sistemas desconectados. LocalLift resuelve ese problema creando un flujo completo: recogida de datos del negocio, generacion de una web premium, captacion de leads, reservas, CRM ligero, panel operativo, tienda, reportes, despliegue y preparacion para integraciones reales con IA, Google, Stripe y email.

La solucion no se limita a ser un generador de webs. Su ambicion es convertirse en un sistema operativo sencillo para negocios locales. El usuario interno de LocalLift puede crear y personalizar la web; el negocio puede operar leads, clientes, reservas, reportes y pedidos; y el cliente final puede consultar informacion, contactar, reservar, comprar o iniciar conversacion desde una web movil primero.

El proyecto actual ya incluye un Studio visual, exportacion HTML, chatbot local, endpoint IA opcional, tienda con backend Stripe de ejemplo, dashboard operativo, CRM, agenda, eventos, reporte mensual, healthcheck, proteccion por token, CORS configurable, despliegue con Docker/Render y exportacion CSV de datos operativos. La base tecnica esta preparada para pilotos online reales, aunque todavia quedan hitos importantes para una plataforma multi-cliente completa: autenticacion por usuarios, base de datos relacional, roles, integraciones productivas con Google, automatizaciones reales de mensajeria, facturacion/presupuestos y escalado operativo.

La propuesta comercial es clara:

LocalLift monta y mantiene todo lo digital de un negocio local: web, Google, reservas, clientes, pagos, tienda, mensajes, reputacion y reportes, sin que el cliente tenga que aprender tecnologia.

El valor diferencial esta en combinar ejecucion tecnica, diseno premium y operacion diaria. Competidores como Wix, Shopify, Odoo, Square o Zoho ofrecen herramientas potentes, pero el pequeno negocio suele necesitar resultado, no software para configurar. LocalLift vende una entrega guiada y mantenida, con una interfaz unica y datos reutilizables.

1. Introduccion

1.1 Contexto del problema

Los negocios locales siguen dependiendo de canales dispersos: Google Maps, Instagram, WhatsApp, llamadas, formularios, reseñas, reservas externas, hojas de calculo y mensajes manuales. Esta dispersion genera varios problemas:

- Informacion duplicada o desactualizada.
- Perdida de leads por falta de seguimiento.
- Reservas gestionadas manualmente y con riesgo de solapes.
- Webs antiguas, lentas o poco convincentes en movil.
- Falta de metricas para justificar el mantenimiento digital.
- Dependencia de plataformas generalistas que el cliente no sabe configurar.
- Poca capacidad para automatizar preguntas frecuentes, recordatorios o solicitudes de reseña.

El pequeno negocio no suele tener un equipo tecnico ni tiempo para aprender una suite completa. Por eso, una solucion viable debe reducir configuracion, guiar la entrega y centrarse en resultados medibles: llamadas, WhatsApps, reservas, formularios, pedidos, pagos y reseñas.

1.2 Motivacion

La motivacion principal del proyecto es crear una herramienta que permita entregar digitalizacion real a negocios locales de forma rapida y repetible. La idea no es crear otro

constructor web generico, sino una plataforma verticalizada para negocios de barrio, clinicas, restaurantes, tiendas, peluqueras, academias, talleres y servicios profesionales.

El proyecto busca resolver tres necesidades al mismo tiempo:

1. Crear webs premium sin empezar desde cero en cada cliente.
2. Gestionar la operacion posterior: leads, reservas, agenda, reportes y datos.
3. Convertir esa operacion en un servicio mensual vendible.

1.3 Enfoque

El enfoque tecnico es incremental. En lugar de construir desde el primer dia una arquitectura pesada con usuarios, microservicios y base relacional, se ha optado por una base sencilla, inspeccionable y rapida:

- Frontend estatico en HTML, CSS y JavaScript.
- Servidor Node nativo, sin framework pesado.
- Persistencia JSON atomica para el MVP.
- Backends de ejemplo para Stripe, OpenAI y Google.
- Documentacion comercial y tecnica desde fases tempranas.
- Despliegue preparado con Render, Docker y healthcheck.

Este enfoque permite validar el producto con pilotos reales antes de hacer inversiones mayores en base de datos, autenticacion avanzada o integraciones complejas.

2. Objetivos del proyecto

2.1 Objetivo general

Desarrollar una plataforma web que permita digitalizar negocios locales mediante un flujo completo de creacion de web, gestion operativa, captacion de clientes, reservas, tienda, analitica, reportes y preparacion para despliegue online.

2.2 Objetivos especificos

- Crear un Studio interno para editar datos, secciones, estilo visual y conversion de una web local.
- Generar una web publica responsive, premium, exportable y lista para hosting.

- Integrar chatbot local configurable con posibilidad de endpoint IA externo.
- Incorporar formulario de lead y captura desde chatbot.
- Registrar eventos de conversion para analitica y reportes.
- Crear una API multi-negocio con persistencia local y backups.
- Crear un portal operativo para el negocio con metricas, leads, clientes, reservas, pedidos, productos, reportes y ajustes.
- Implementar CRM ligero con estados, notas, actividades y exportacion CSV.
- Implementar agenda MVP con servicios, disponibilidad, bloqueos, reservas, validacion de solapes y recordatorios manuales.
- Crear reporte mensual operativo con recomendaciones accionables.
- Preparar despliegue en cloud con healthcheck, token admin, CORS configurable, Docker, Render Blueprint y validacion de entorno.
- Documentar la propuesta comercial, el guion de demo, los paquetes y el roadmap.

2.3 Criterios de exito

El proyecto se considera exitoso en su fase actual si:

- Un negocio puede tener una web profesional funcional en poco tiempo.
- El visitante puede llamar, abrir mapa, enviar lead, usar chatbot o pedir reserva.
- El lead o reserva puede aparecer en el dashboard.
- El operador puede cambiar estados, guardar notas, preparar recordatorios y exportar datos.
- El reporte mensual puede mostrar valor con metricas comprensibles.
- El backend puede desplegarse online con healthcheck y seguridad minima.
- La solucion puede explicarse y venderse con un guion claro.

3. Alcance actual

3.1 Lo que incluye el MVP avanzado

El proyecto actual incluye:

- Studio generador de webs.
- Demos por sector.
- Vista previa desktop, tablet y movil.

- Motor visual con temas, colores, tipografia, densidad, formas e interacciones.
- Panel de cambios rapidos para demos con cliente.
- Undo/redo de variantes.
- Exportacion HTML standalone con CSS y JS integrados.
- Import/export JSON de datos del negocio.
- Chatbot local configurable.
- Endpoint IA opcional.
- Formulario de lead.
- Widget publico de reservas.
- Tienda online visual con carrito.
- Backend de ejemplo Stripe con pedidos, stock, cupones y webhooks.
- Backend de ejemplo Google.
- Backend de ejemplo chatbot IA.
- API multi-negocio.
- CRM con contactos, estados, actividades y notas.
- Reservas con disponibilidad, bloqueos y validacion de solapes.
- Eventos de conversion persistentes.
- Dashboard operativo.
- Reporte mensual HTML imprimible.
- Exportacion CSV de leads, clientes y reservas.
- Healthcheck.
- Token admin opcional para produccion.
- CORS configurable.
- Dockerfile.
- Render Blueprint.
- `.env.example`.
- Guia de despliegue.
- Runbook de piloto online.
- Documentacion comercial e inversores.

3.2 Lo que no incluye todavia

Aunque el proyecto esta muy avanzado para una demo vendible, todavia no incluye:

- Autenticacion completa por usuarios y contraseñas.
- Roles reales por usuario.

- Base de datos relacional o SQLite.
- Panel interno multi-cliente completo con usuarios.
- Envio automatico real de WhatsApp/email.
- OAuth Google por negocio.
- Conexion productiva a Google Business Profile.
- Facturacion y presupuestos.
- PDF automatico de reportes desde servidor.
- Subida de imagenes a storage externo.
- Notificaciones push o bandeja de mensajes completa.
- Integracion productiva del chatbot IA multi-negocio con base de conocimiento persistente.
- Sistema de facturacion recurrente del propio SaaS.

4. Vision de producto

4.1 Producto final deseado

LocalLift debe evolucionar hacia una plataforma de digitalizacion total para negocios locales. La version final deberia cubrir tres capas:

1. **Studio interno:** herramienta usada por LocalLift para crear, editar, publicar y mantener clientes.
2. **Portal del negocio:** panel usado por el dueño o equipo del negocio para operar su día a día.
3. **Web publica:** experiencia para visitantes finales: contacto, reserva, compra, mapa, chatbot y confianza.

4.2 Principios de diseno del producto

- **Movil primero:** la mayoria de interacciones locales nacen desde movil.
- **Menos configuracion:** presets, plantillas y campos guiados.
- **Datos reutilizables:** una vez introducido un dato, debe alimentar web, bot, reportes y CRM.
- **Verticalizacion:** no vender software generico, sino soluciones por sector.
- **Portabilidad:** exportar web, datos y CSV para evitar dependencia opaca.
- **Medicion de valor:** reportes y conversiones como base del mantenimiento mensual.
- **Simplicidad operativa:** evitar construir un ERP pesado.

4.3 Propuesta de valor

Para el negocio local:

- Web profesional sin aprender una herramienta.
- Mas confianza en Google, movil y redes.
- Menos preguntas repetidas gracias al chatbot.
- Leads y reservas centralizados.
- Reporte mensual con resultados.
- Servicio mantenido y evolucionable.

Para LocalLift:

- Proceso repetible de entrega.
- Base para mantenimiento mensual.
- Capacidad de gestionar varios negocios.
- Paquetes vendibles por sector.
- Diferenciacion frente a constructores web tradicionales.

5. Analisis competitivo

5.1 Wix, Squarespace, Webflow y Framer

Estos productos resuelven la creacion visual de webs, pero suelen exigir que el cliente escriba, estructure, configure SEO, conecte formularios, optimice movil, mantenga contenidos y entienda la herramienta. LocalLift no compite solo en editor visual, sino en resultado entregado y gestion posterior.

Ventaja de LocalLift:

- El cliente no tiene que aprender el editor.
- La web nace con formularios, reservas, chatbot y datos operativos.
- El resultado esta pensado para negocios locales, no para cualquier caso generico.
- El mantenimiento mensual se justifica con reportes.

5.2 Shopify y Square

Shopify y Square son fuertes en comercio y pagos, pero no cubren de forma integrada la presencia local, Google, reputacion, reservas, chatbot y mantenimiento para negocios no puramente ecommerce.

Ventaja de LocalLift:

- Une web local, tienda ligera, reservas y CRM.
- Prioriza llamadas, WhatsApp, mapa y reservas.
- Puede ser mas simple para restaurantes, clinicas, peluquerias o servicios.

5.3 Odoo y Zoho

Odoo y Zoho son suites muy potentes, pero pueden resultar excesivas para negocios pequeños. El coste no esta solo en la licencia, sino en implantacion, configuracion, curva de aprendizaje y soporte.

Ventaja de LocalLift:

- Interfaz mucho mas acotada.
- Modulos preseleccionados.
- Entrega hecha por el proveedor.
- Menor friccion de adopcion.

5.4 Chatbots y helpdesks

Herramientas como Crisp, Intercom o soluciones basadas en WhatsApp Business ofrecen mensajería y automatización, pero no necesariamente crean toda la presencia digital del negocio.

Ventaja de LocalLift:

- El chatbot utiliza los datos de la web y del negocio.
- La captura de lead se integra con CRM y reportes.
- La automatización se enmarca en una entrega completa.

6. Arquitectura general

6.1 Vista de alto nivel

La arquitectura se divide en:

- **Frontend principal:** `index.html`, `src/app.js`, `src/styles.css`.
- **Paginas auxiliares:** `pages/`.
- **Servidor Node:** `server/server.mjs`.
- **APIs modulares:** `server/api/`.
- **Librerias de servidor:** `server/lib/`.
- **Ejemplos de integracion:** `examples/`.
- **Datos JSON:** `data/`.
- **Documentacion:** `docs/`.
- **Recursos vendor:** `assets/vendor/`.

6.2 Frontend

El frontend esta construido sin framework pesado. Esto reduce dependencias, facilita abrir el proyecto directamente y simplifica la exportacion HTML. La aplicacion principal maneja:

- Estado del negocio.
- Formularios de edicion.
- Render de vista previa.
- Exportacion.
- Import/export JSON.
- Chatbot local.
- Tienda visual.
- Eventos.
- Integracion con API si el servidor esta activo.

6.3 Backend

El backend usa Node nativo y modulos `.mjs`. El servidor sirve archivos estaticos y delega rutas API a modulos especializados:

- `business-api.mjs`
- `contact-api.mjs`
- `booking-api.mjs`
- `event-api.mjs`
- `report-api.mjs`
- `health-api.mjs`

El diseño es deliberadamente sencillo para facilitar lectura, modificacion y despliegue.

6.4 Persistencia

La persistencia principal del MVP usa `data/business-db.json`. El acceso se centraliza con `server/lib/json-store.mjs`, que implementa:

- Lectura con fallback.
- Escritura atomica.
- Creacion de directorios.
- Backups antes de cambios importantes.

Este enfoque es suficiente para desarrollo, demos y pilotos controlados. Para escalar, se recomienda SQLite primero y Postgres despues.

6.5 Despliegue

El proyecto esta preparado para:

- Ejecucion local con `npm.cmd start`.
- Produccion con `npm run start:prod`.
- Validacion de entorno con `npm run check:deploy`.
- Render Blueprint con `render.yaml`.
- Contenedor con `Dockerfile`.
- Healthcheck en `/api/health`.
- Frontend separado mediante `?apiBase=...`.

7. Estructura de carpetas

```
.
├── index.html
├── package.json
├── Dockerfile
├── render.yaml
├── .env.example
├── src/
│   ├── app.js
│   ├── styles.css
│   ├── business/
│   │   ├── dashboard.js
│   │   └── monthly-report.js
│   ├── shared/
│   │   └── api-config.js
│   └── styles/
│       ├── business.css
│       └── report.css
├── pages/
│   ├── business-dashboard.html
│   ├── monthly-report.html
│   ├── store-admin.html
│   ├── onboarding.html
│   ├── google-ops.html
│   └── investor.html
├── server/
│   ├── server.mjs
│   ├── api/
│   ├── lib/
│   └── scripts/
├── examples/
├── assets/vendor/
├── data/
└── docs/
```

7.1 Archivos principales

- `index.html` : entrada del Studio.
- `src/app.js` : logica principal de edicion, preview, exportacion, chatbot, tienda y sincronizacion.
- `src/styles.css` : estilos del Studio y webs generadas.
- `pages/business-dashboard.html` : portal operativo.
- `src/business/dashboard.js` : logica del portal.
- `src/styles/business.css` : estilos del portal.

- `pages/monthly-report.html` : reporte mensual imprimible.
 - `src/business/monthly-report.js` : logica del reporte.
 - `server/server.mjs` : servidor estatico y router API.
 - `server/lib/admin-auth.mjs` : token admin opcional.
 - `server/lib/cors.mjs` : CORS configurable.
 - `server/lib/json-store.mjs` : persistencia atomica JSON.
 - `server/scripts/validate-deploy-env.mjs` : validacion de entorno productivo.
-

8. Funcionalidades del Studio

8.1 Editor de datos del negocio

El Studio permite editar:

- Nombre.
- Categoria.
- Ciudad/ubicacion.
- Descripcion.
- Propuesta de valor.
- Telefono.
- Email.
- Direccion.
- Servicios.
- Diferenciales.
- Horarios.
- Galeria.
- Testimonios.
- FAQ.
- Enlaces.
- CTA principal.
- Objetivo de conversion.
- Datos de Google.
- Datos de tienda.

- Datos de chatbot.

El formato de entrada esta optimizado para rapidez: listas por linea y estructuras simples como

Nombre | Texto .

8.2 Vista previa responsive

El Studio permite alternar entre:

- Desktop.
- Tablet.
- Movil.

La vista movil es especialmente importante porque los negocios locales reciben buena parte de sus visitas desde smartphone.

8.3 Temas visuales

Incluye cinco direcciones visuales:

- Aurora.
- Carbon.
- Editorial.
- Neon.
- Luxe.

Cada tema modifica sensacion visual, color, contraste y estilo de marca, permitiendo adaptar el resultado a diferentes sectores.

8.4 Personalizacion avanzada

El Studio permite cambiar:

- CTA.
- Titulares de seccion.
- Tipografia.
- Densidad.
- Forma visual.
- Color.
- Animacion.
- Secciones visibles.

8.5 Demo en vivo

El panel de demo en vivo permite aplicar cambios rapidos sin navegar por todos los campos. Incluye:

- Variantes comerciales.
- Comando rapido.
- Toggles de secciones.
- Undo/redo.

Ejemplos de comandos:

- `mas premium`
- `vender reservas`
- `sin mapa`
- `sin bot`
- `mostrar todo`

Esta funcionalidad esta pensada para vender delante del cliente, mostrando transformaciones visibles en segundos.

8.6 Checklist de calidad

El Studio calcula un score de entrega y muestra una checklist de calidad. Esto permite saber si la web esta lista para presentar o publicar. La checklist cubre elementos como:

- CTA.
- Servicios.
- Fotos.
- Testimonios.
- FAQ.
- Contacto.
- Google.
- Formulario.
- Chatbot.
- Tienda o reservas si aplica.

8.7 Exportacion HTML

La exportacion crea un HTML standalone con:

- Contenido del negocio.
- CSS integrado.
- JavaScript integrado.
- Recursos vendor integrados cuando estan disponibles.
- Schema LocalBusiness.
- Meta title y description.
- Formularios y widgets funcionales.
- Fallback local para leads y reservas.

Si el Studio tiene configurado `apiBase`, el HTML exportado incluye:

```
<meta name="locallift-api-base" content="https://tu-api.com">
```

Esto permite que una web estatica publicada en otro hosting envíe leads, reservas y eventos al backend online.

9. Web publica generada

9.1 Secciones principales

La web generada puede incluir:

- Hero con imagen grande.
- Propuesta de valor.
- Servicios.
- Prueba rapida / confianza.
- Galeria.
- Diferenciales.
- Horario.
- Testimonios.
- FAQ.
- Mapa.
- Formulario de lead.
- Widget de reserva.

- Tienda o catalogo.
- Chatbot.
- Dock flotante de conversion.
- Footer.

9.2 Conversion

La web esta orientada a acciones concretas:

- Llamar.
- Abrir WhatsApp.
- Reservar.
- Comprar.
- Ver mapa.
- Enviar formulario.
- Abrir chatbot.
- Solicitar informacion.

9.3 Fallback standalone

Si la web exportada no tiene backend disponible:

- Los leads se guardan en `window.localLiftLeads`.
- Las reservas se guardan en `window.localLiftBookings`.
- Los eventos se guardan en `window.localLiftEvents`.

Esto evita que la web deje de funcionar por completo y mantiene una experiencia aceptable en demos o contextos sin servidor.

9.4 Analitica de eventos

Eventos actuales:

- `booking_click`
- `lead_form_submit`
- `store_add_to_cart`
- `store_checkout_start`
- `chatbot_open`
- `chatbot_prompt`

- chatbot_message
- chatbot_lead_captured
- public_booking_submit
- dock_booking_click
- dock_phone_click
- dock_maps_click

Si el backend esta activo, estos eventos se sincronizan con la API para alimentar reportes.

10. Chatbot

10.1 Chatbot local

El bot local responde usando los datos del negocio:

- Servicios.
- Horarios.
- Ubicacion.
- Telefono.
- Email.
- FAQ.
- Enlaces.
- Reservas.

No depende de una API externa y funciona incluso en el HTML exportado.

10.2 Configuracion

El usuario puede configurar:

- Nombre del asistente.
- Tono.
- Saludo inicial.
- Preguntas rapidas.
- Texto de traspaso a humano.
- Endpoint IA opcional.

10.3 Captura de leads

Cuando el visitante escribe datos de contacto o expresa una necesidad, el chatbot puede generar un lead. Si hay backend, lo guarda en CRM; si no, lo conserva en memoria local.

10.4 Endpoint IA opcional

El proyecto incluye `examples/chatbot-api.example.mjs`, un backend de ejemplo que llama a la API de OpenAI desde servidor. La clave nunca debe exponerse en HTML.

Variables:

```
OPENAI_API_KEY
OPENAI_MODEL
```

10.5 Limitaciones actuales

- No existe todavía base de conocimiento persistente por negocio.
- No hay historial de conversaciones productivo.
- No hay agente que ejecute acciones reales con confirmación.
- No hay panel de entrenamiento de FAQ ampliada.

11. CRM y leads

11.1 Objetivo

El CRM ligero busca que ningún contacto se pierda. Un negocio local no necesita un CRM complejo al principio; necesita saber quien ha preguntado, por que canal, en que estado esta y que tarea queda pendiente.

11.2 Modelo de contacto

Campos principales:

- `id`
- `businessId`
- `type`
- `name`

- phone
- email
- source
- status
- tags
- notes
- valueEstimate
- lastInteractionAt
- createdAt
- updatedAt

11.3 Estados

Estados soportados:

- new
- contacted
- waiting
- reserved
- won
- lost
- customer

11.4 Actividades

Cada contacto puede tener actividades:

- Lead creado.
- Contacto creado manualmente.
- Nota.
- Tarea.
- Cambio de estado.
- Reserva relacionada.
- Recordatorio preparado.

11.5 Endpoints

```
POST /api/public/{businessSlug}/leads
GET /api/businesses/{id}/contacts?includeActivities=true
POST /api/businesses/{id}/contacts
PATCH /api/businesses/{id}/contacts/{contactId}
POST /api/businesses/{id}/contacts/{contactId}/activities
```

11.6 Portal

El dashboard muestra los leads en pipeline por estado. Permite:

- Ver leads nuevos.
- Cambiar estado.
- Guardar notas/tareas.
- Consultar clientes.
- Exportar leads a CSV.
- Exportar clientes a CSV.

11.7 CSV

Los CSV incluyen:

- Nombre.
- Telefono.
- Email.
- Estado.
- Fuente.
- Etiquetas.
- Valor.
- Notas.
- Ultima actividad.
- Fecha de creacion.

Esto refuerza el principio de portabilidad y permite usar los datos en Excel, Google Sheets o herramientas externas.

12. Reservas y agenda

12.1 Objetivo

Permitir que negocios con citas puedan recibir reservas desde su web y gestionarlas desde el portal sin una herramienta externa compleja.

12.2 Servicios reservables

Cada servicio puede incluir:

- Nombre.
- Duracion.
- Precio.
- Estado activo/inactivo.

Si un negocio no tiene servicios reservables, la API puede generar una base inicial desde los servicios ya definidos en la web.

12.3 Disponibilidad semanal

El dashboard permite editar tramos por día de la semana:

- Día.
- Activo.
- Hora inicio.
- Hora fin.

12.4 Bloqueos manuales

Sirven para cerrar huecos concretos:

- Vacaciones.
- Evento.
- Cierre temporal.
- Descanso.
- Mantenimiento.

12.5 Validacion de reservas

La API evita:

- Reservas fuera de horario.
- Solapes con otras reservas.
- Reservas en bloqueos activos.
- Negocios archivados.
- Servicios inactivos.

12.6 Recordatorios

El sistema permite:

- Preparar recordatorio manual por reserva.
- Generar cola de proximos recordatorios en 48 horas.
- Registrar la accion en CRM.
- Devolver enlaces preparados para WhatsApp/email.

Todavia no envia automaticamente. El envio automatico real queda como siguiente hito.

12.7 Vista calendario

El dashboard incluye calendario semanal:

- Semana anterior.
- Hoy.
- Semana siguiente.
- Reservas por dia.
- Estado.
- Servicio.
- Cliente.
- Recordatorio.

12.8 Exportacion CSV

Las reservas se pueden exportar con:

- Servicio.
- Cliente.
- Telefono.
- Email.
- Contacto.
- Estado.

- Inicio.
- Fin.
- Notas.
- Ultimo recordatorio.
- Creado.

13. Tienda, pagos y pedidos

13.1 Tienda visual en la web

El Studio permite activar tienda con:

- Productos.
- Imagen.
- Precio.
- Descripcion.
- SKU.
- Stock.
- Carrito.
- Checkout.

13.2 Backend de comercio

El archivo `examples/commerce-api.example.mjs` implementa una base de backend de tienda con:

- Productos.
- Configuracion.
- Validacion de carrito.
- Checkout Stripe.
- Webhook Stripe.
- Pedidos.
- Admin.
- Cupones.
- Ajustes.

- Stock reservado.
- Emails opcionales con Resend.

13.3 Seguridad del backend de comercio

Incluye:

- Libreria oficial `stripe`.
- Verificacion de webhook con `Stripe-Signature`.
- `STORE_ADMIN_TOKEN` obligatorio.
- Comparacion en tiempo constante.
- CORS configurable.
- Redirecciones limitadas.
- Rate limit.
- JSON estricto.
- Escritura atomica.

13.4 Limitaciones actuales

La tienda aun debe consolidarse con el portal multi-negocio principal. Falta:

- Variantes.
- Categorias.
- Inventario avanzado.
- POS ligero.
- Cierre diario.
- Pedidos unificados en dashboard principal.

14. Reportes y analitica

14.1 Objetivo

El reporte mensual debe demostrar valor al cliente de forma comprensible. No debe ser un panel tecnico de analitica, sino un resumen operativo:

- Contactos nuevos.
- Reservas.

- Recordatorios.
- Ingresos.
- Fuentes.
- Conversiones.
- Recomendaciones.

14.2 API de reporte

Endpoint:

```
GET /api/businesses/{id}/reports/monthly?month=YYYY-MM
```

Si no se pasa mes, usa el mes actual.

14.3 Vista imprimible

La vista vive en:

```
pages/monthly-report.html
```

Permite imprimir desde navegador y generar un PDF manual. Falta todavia automatizar PDF desde servidor o pipeline.

14.4 Recomendaciones automaticas

El reporte genera recomendaciones basicas segun datos:

- Responder leads.
- Activar reservas.
- Revisar conversiones.
- Mejorar fuentes.
- Preparar recordatorios.

14.5 Importancia comercial

El reporte es clave para cobrar mantenimiento mensual. Permite decir:

- "Este mes recibiste X contactos."
- "La accion mas usada fue WhatsApp."

- "Tienes reservas sin recordatorio."
- "Conviene subir fotos o activar promocion."

15. Google Ops y SEO local

15.1 Campos Google-ready

El Studio contempla:

- Google Maps URL.
- Mapa embebido.
- Nota de llegada.
- Parking.
- Enlace de reseñas.
- URL de reserva.
- Place ID.
- Datos NAP.

15.2 Backend Google de ejemplo

`examples/google-integration.example.mjs` cubre:

- Places API.
- Calendar FreeBusy.
- Calendar Events.
- Business Profile accounts.
- Business Profile locations.
- Business Profile reviews.
- Reply a reseñas.
- Workspace user dry-run.
- Review request.

15.3 Playbook Google

La documentacion incluye un playbook para vender y operar:

- Google Business Profile.
- Correo profesional.
- Reseñas.
- Reservas.
- Datos de negocio.

15.4 Pendiente

Para produccion falta:

- OAuth persistente por negocio.
- Guardar tokens cifrados.
- Solicitar acceso Business Profile API.
- Gestion de permisos.
- Sincronizacion real de horarios/fotos/reseñas.

16. Seguridad

16.1 Estado actual

Seguridad implementada:

- Token admin opcional para rutas `/api/businesses/*`.
- Soporte `Authorization: Bearer`.
- Soporte `X-LocalLift-Admin-Token`.
- CORS configurable.
- Healthcheck separado.
- Validacion de entorno productivo.
- Escritura atomica.
- Backups JSON.
- Proteccion basica contra path traversal en servidor estatico.
- Cabeceras base como `nosniff`, `Referrer-Policy` y `Cache-Control: no-store`.

16.2 Validacion de entorno

`server/scripts/validate-deploy-env.mjs` impide arrancar `start:prod` si faltan:

- `NODE_ENV=production`
- `HOST=0.0.0.0`
- Token admin largo.
- `CORS_ORIGIN` HTTPS.
- `BUSINESS_DB_FILE`.

16.3 Limitaciones

Falta:

- Usuarios reales.
- Sesiones.
- Hash de contraseñas.
- Roles.
- Permisos por negocio.
- Auditoria avanzada.
- Rate limit general en API principal.
- Proteccion CSRF si se introducen sesiones con cookies.
- Cifrado de tokens Google.

17. Despliegue

17.1 Local

```
npm.cmd start
```

Servidor:

```
http://127.0.0.1:5173
```

17.2 Produccion

```
npm run start:prod
```

Variables minimas:

```
NODE_ENV=production
HOST=0.0.0.0
LOCALLIFT_ADMIN_TOKEN=<token-largo>
CORS_ORIGIN=https://www.cliente.com,https://cliente.com
BUSINESS_DB_FILE=/data/business-db.json
BUSINESS_DB_BACKUP_DIR=/data/backups
BUSINESS_DB_BACKUPS=true
```

17.3 Render

El repo incluye `render.yaml`:

- Servicio web Node.
- Build command `npm ci --omit=dev`.
- Start command `npm run start:prod`.
- Healthcheck `/api/health`.
- Disco persistente `/data`.
- Variables secretas pendientes de rellenar.

17.4 Docker

El `Dockerfile` usa:

- `node:22-alpine`
- `npm ci --omit=dev`
- `HOST=0.0.0.0`
- `BUSINESS_DB_FILE=/data/business-db.json`
- Healthcheck con `fetch('/api/health')`.

17.5 Frontend separado

Si la web esta en un dominio y la API en otro:

```
https://www.cliente.com/pages/business-dashboard.html?apiBase=https://tu-api.onrender
```

La URL API puede resolverse por:

1. `window.LOCALLIFT_API_BASE`

2. Meta `locallift-api-base`
3. `localStorage.locallift_api_base`
4. Mismo dominio

18. Guion de defensa tecnica tipo TFG

18.1 Apertura

"Este proyecto presenta LocalLift Studio, una plataforma para digitalizar negocios locales mediante una solucion integrada de web, CRM, reservas, tienda, chatbot, analitica y reportes. El objetivo ha sido crear una herramienta capaz de pasar de datos basicos de un negocio a una presencia digital operativa, exportable y preparada para pilotos reales."

18.2 Problema

"Los negocios locales suelen tener presencia digital fragmentada: redes sociales, Google Maps, WhatsApp, reservas manuales y webs antiguas. Esto provoca perdida de oportunidades y dificultad para medir resultados. El proyecto aborda este problema simplificando la creacion y gestion digital."

18.3 Solucion

"La solucion se divide en tres capas: un Studio interno para crear y personalizar webs, una web publica orientada a conversion y un portal operativo para gestionar leads, clientes, reservas y reportes. La arquitectura usa frontend estatico, servidor Node modular y persistencia JSON atomica para validar rapidamente."

18.4 Demo tecnica

1. Abrir `index.html`.
2. Mostrar editor de negocio.
3. Cambiar un dato visible.
4. Mostrar preview movil.
5. Aplicar una variante de demo en vivo.
6. Activar formulario de lead.
7. Exportar HTML.
8. Abrir dashboard.

9. Mostrar leads, reservas y reportes.
10. Mostrar `/api/health`.
11. Explicar despliegue con Render y Docker.

18.5 Decisiones de arquitectura

"Se eligio JavaScript sin framework pesado para mantener portabilidad y exportacion HTML. Se eligio Node nativo para reducir dependencias. Se eligio JSON atomico para acelerar el MVP, con una ruta clara de migracion a SQLite/Postgres."

18.6 Seguridad

"La seguridad actual cubre token admin, CORS configurable, validacion de entorno de produccion y separacion basica por negocio. La autenticacion completa por usuarios queda como trabajo futuro."

18.7 Resultados

"El resultado es una plataforma funcional que ya permite crear webs, captar leads, gestionar CRM, recibir reservas, exportar CSV, generar reportes y desplegar backend. Esto cumple una fase avanzada de prototipo/MVP."

18.8 Cierre

"La conclusion es que LocalLift demuestra viabilidad tecnica y comercial. El siguiente paso no es seguir añadiendo funcionalidades aisladas, sino validar con pilotos reales y, a partir de ahi, evolucionar la persistencia, autenticacion e integraciones."

19. Guion comercial de demo

19.1 Entrada

"Ahora mismo tus clientes te descubren desde Google, Instagram o WhatsApp. Si cuando entran no ven horarios claros, fotos buenas, reserva facil y respuestas rapidas, se van a otro negocio. LocalLift te deja todo eso montado."

19.2 Demo movil primero

1. Mostrar web en movil.

2. Enseñar CTA principal.
3. Abrir mapa.
4. Abrir reserva.
5. Abrir chatbot.
6. Enviar lead de prueba.

19.3 Mostrar operacion

1. Abrir dashboard.
2. Ver lead nuevo.
3. Cambiar estado.
4. Añadir nota.
5. Ver calendario de reservas.
6. Preparar recordatorio.
7. Abrir reporte mensual.

19.4 Cierre

"No te vendemos una plantilla. Te entregamos una presencia digital viva: web, captacion, reservas, seguimiento y reporte. Tu negocio no solo se ve mejor; tambien sabe que contactos y reservas ha generado."

19.5 Objeciones

Ya tengo Instagram. Instagram ayuda, pero no organiza servicios, horarios, reservas, SEO, FAQ, mapa y reportes en una experiencia controlada.

Wix es mas barato. Wix es una herramienta. LocalLift vende resultado implementado, conectado y mantenido.

No necesito chatbot. El bot reduce preguntas repetidas y deriva a humano cuando hace falta.

No quiero depender. La web se exporta y los datos se pueden sacar en JSON/CSV.

20. Guion para inversores

20.1 Tesis

"Existe una oportunidad en digitalizar negocios locales con una solución más simple que un ERP y más operativa que un constructor web. LocalLift combina entrega de web, CRM, reservas, Google, chatbot, reportes y mantenimiento."

20.2 Mercado

El mercado objetivo inicial son negocios locales que dependen de visitas, llamadas, reservas o confianza:

- Restaurantes.
- Clínicas.
- Peluquerías.
- Barberías.
- Gimnasios.
- Academias.
- Talleres.
- Tiendas locales.
- Servicios profesionales.

20.3 Producto

Producto en tres capas:

- Studio interno.
- Portal del negocio.
- Web pública.

20.4 Modelo de negocio

Paquetes:

- Presencia Local.
- Operación Local.
- Comercio Local.
- Digitalización Total.

Ingresos:

- Setup inicial.
- Mantenimiento mensual.
- Integraciones premium.

- Cambios mensuales.
- Reportes.
- Servicios Google.

20.5 Traccion tecnica

Se ha construido:

- Motor web.
- CRM.
- Reservas.
- Reportes.
- Tienda base.
- Despliegue.
- Documentacion.

20.6 Siguiente hito

Cerrar 5 pilotos fundadores, publicar 3 demos reales y medir conversiones utiles por mes.

21. Modelo comercial

21.1 Paquete Presencia Local

Para negocios que necesitan verse profesionales.

Incluye:

- Web premium.
- SEO local basico.
- WhatsApp/telefono/mapa.
- Chatbot local.
- Formulario.
- Reporte simple.

21.2 Paquete Operacion Local

Para negocios con leads o reservas.

Incluye:

- Todo Presencia Local.
- CRM.
- Agenda.
- Reservas.
- Reporte mensual.
- Solicitud de reseñas.

21.3 Paquete Comercio Local

Para negocios con productos.

Incluye:

- Web.
- Tienda.
- Carrito.
- Checkout.
- Pedidos.
- Inventario basico.
- Cupones.

21.4 Paquete Digitalizacion Total

Para negocios que quieren delegarlo casi todo.

Incluye:

- Web.
- CRM.
- Reservas.
- Tienda o presupuestos.
- Google.
- Reportes.
- Marketing.
- Reputacion.
- IA.

- Soporte prioritario.

22. Roadmap pendiente

22.1 Prioridad inmediata

- Desplegar backend real en Render/Railway/VPS.
- Publicar 2-3 demos reales con URL.
- Probar embudo completo con `apiBase`.
- Cerrar pilotos fundadores.
- Medir conversiones utiles.

22.2 Prioridad tecnica alta

- Migrar de JSON a SQLite o Supabase/Postgres.
- Crear autenticacion real.
- Crear roles y permisos.
- Crear usuarios por negocio.
- Cifrar tokens externos.
- Mejorar auditoria.
- Añadir pruebas automatizadas.

22.3 Prioridad de producto

- Bandeja de mensajes.
- Automatizaciones.
- Solicitudes de reseña.
- Marketing local.
- Presupuestos.
- Facturas.
- Exportacion PDF automatica de reportes.
- Subida de imagenes.

22.4 Integraciones

- OAuth Google.
- Google Calendar real.
- Google Business Profile.
- Email transaccional.
- WhatsApp Business API.
- Stripe productivo.
- Storage externo.

23. Riesgos

23.1 Riesgo tecnico: JSON en produccion

El JSON es rapido para MVP, pero no ideal para concurrencia. Mitigacion: pilotos controlados y migracion posterior a SQLite/Postgres.

23.2 Riesgo comercial: vender demasiado pronto

Mitigacion: vender pilotos fundadores con expectativas claras y alcance limitado.

23.3 Riesgo de soporte

Si cada cliente requiere ajustes manuales infinitos, el margen baja. Mitigacion: plantillas, checklists, procesos y limites por plan.

23.4 Riesgo de integraciones

Google, WhatsApp y pagos pueden requerir permisos, aprobaciones y configuracion. Mitigacion: empezar con enlaces y dry-run, y activar APIs solo cuando haya necesidad real.

23.5 Riesgo de seguridad

El MVP no tiene usuarios completos. Mitigacion: token admin, CORS, despliegue controlado y desarrollo de autenticacion antes de escalar.

24. Plan de validacion

24.1 Pruebas tecnicas

- `npm.cmd run check`.
- `npm.cmd run check:deploy`.
- `GET /api/health`.
- Prueba de CORS.
- Prueba de token admin.
- Crear lead publico.
- Crear reserva publica.
- Cambiar estado de lead.
- Crear nota.
- Confirmar/cancelar reserva.
- Exportar CSV.
- Abrir reporte mensual.

24.2 Pruebas UX

- Revisar movil 360px.
- Revisar tablet.
- Revisar desktop.
- Comprobar que botones no se solapan.
- Comprobar formularios.
- Comprobar textos largos.
- Comprobar estados vacios.

24.3 Pruebas comerciales

- Mostrar demo en menos de 10 minutos.
- Explicar precio y paquete.
- Responder objeciones.
- Cerrar siguiente paso.

25. Manual de uso rapido

25.1 Crear web

1. Abrir `index.html`.
2. Rellenar datos.
3. Elegir tema.
4. Activar secciones.
5. Revisar preview movil.
6. Revisar checklist.
7. Exportar HTML.

25.2 Gestionar negocio

1. Ejecutar servidor.
2. Abrir `pages/business-dashboard.html`.
3. Seleccionar negocio.
4. Revisar metricas.
5. Gestionar leads.
6. Gestionar reservas.
7. Abrir reporte.

25.3 Desplegar

1. Subir repo a GitHub.
2. Usar Render Deploy.
3. Configurar variables.
4. Comprobar `/api/health`.
5. Abrir dashboard con `apiBase`.
6. Probar lead/reserva/reporte.

26. Estado actual por porcentaje

26.1 Demo vendible online

Estado estimado: 94-95%.

Falta:

- Crear servicio real en Render/Railway.
- Configurar variables reales.
- Publicar frontend/demo.
- Probar el embudo completo en dominio real.
- Documentar URLs definitivas.

26.2 Producto completo Plan Maestro

Estado estimado: 75-76%.

Falta:

- Autenticacion real.
- Base relacional.
- Integraciones productivas.
- Mensajeria automatica.
- Facturacion/presupuestos.
- Marketing y reputacion.
- Multiusuario.
- Escalado.

27. Conclusiones

LocalLift Studio ha evolucionado desde un generador de webs premium hasta una base real de plataforma operativa para negocios locales. El proyecto ya no se limita a mostrar una web bonita: captura leads, gestiona CRM, recibe reservas, registra eventos, genera reportes, exporta datos y se prepara para despliegue en cloud.

La decision de construir de forma incremental ha permitido avanzar rapido y mantener el sistema comprensible. El uso de JSON, Node nativo y frontend sin framework pesado no pretende ser la arquitectura final, sino una estrategia adecuada para validar producto, vender pilotos y aprender con clientes reales.

El siguiente gran salto es comercial y operativo: publicar demos reales, cerrar pilotos y medir conversiones utiles. A partir de ahi, la prioridad tecnica debe ser convertir la base MVP en plataforma robusta: autenticacion, base relacional, roles, integraciones y automatizaciones.

Como memoria de proyecto, LocalLift demuestra:

- Identificacion de un problema real.
- Solucion tecnica funcional.
- Enfoque comercial claro.
- Arquitectura evolutiva.
- Documentacion y despliegue.
- Roadmap realista.

El proyecto esta en una fase muy cercana a una demo vendible online y con una ruta clara hacia producto SaaS/servicio gestionado para negocios locales.

Anexo A. Endpoints principales

```
GET    /api/health
GET    /api/businesses
POST   /api/businesses
GET    /api/businesses/{id}
PUT    /api/businesses/{id}
DELETE /api/businesses/{id}/archive

POST   /api/public/{businessSlug}/leads
GET    /api/businesses/{id}/contacts
POST   /api/businesses/{id}/contacts
PATCH /api/businesses/{id}/contacts/{contactId}
POST   /api/businesses/{id}/contacts/{contactId}/activities

GET    /api/businesses/{id}/services
POST   /api/businesses/{id}/services
PATCH /api/businesses/{id}/services/{serviceId}

GET    /api/businesses/{id}/bookings
POST   /api/businesses/{id}/bookings
PATCH /api/businesses/{id}/bookings/{bookingId}
POST   /api/businesses/{id}/bookings/{bookingId}/reminders

GET    /api/businesses/{id}/availability
PUT    /api/businesses/{id}/availability

GET    /api/businesses/{id}/blocks
POST   /api/businesses/{id}/blocks
PATCH /api/businesses/{id}/blocks/{blockId}

GET    /api/businesses/{id}/reminders
POST   /api/businesses/{id}/reminders

POST   /api/public/{businessSlug}/bookings
POST   /api/public/{businessSlug}/events
GET    /api/businesses/{id}/events

GET    /api/businesses/{id}/reports/monthly
```


Anexo B. Variables de entorno

```
PORT=5173
HOST=127.0.0.1
NODE_ENV=development
LOCALLIFT_ADMIN_TOKEN=change-me-use-a-long-random-token
CORS_ORIGIN=http://127.0.0.1:5173,http://localhost:5173
BUSINESS_DB_FILE=data/business-db.json
BUSINESS_DB_BACKUPS=true
OPENAI_API_KEY=
OPENAI_MODEL=gpt-5-mini
STORE_ADMIN_TOKEN=change-me-use-a-long-random-token
CHECKOUT_ALLOWED_ORIGINS=http://127.0.0.1:5173,http://localhost:5173
STRIPE_SECRET_KEY=
STRIPE_WEBHOOK_SECRET=
ORDER_EMAIL_TO=
RESEND_API_KEY=
GOOGLE_MAPS_API_KEY=
GOOGLE_CALENDAR_ACCESS_TOKEN=
GOOGLE_CALENDAR_ID=primary
GOOGLE_BUSINESS_PROFILE_ACCESS_TOKEN=
GOOGLE_WORKSPACE_ADMIN_ACCESS_TOKEN=
```

Anexo C. Checklist de entrega a cliente

- Web revisada en movil.
- Web revisada en desktop.
- CTA principal visible.
- Telefono correcto.
- WhatsApp correcto.
- Mapa correcto.
- Horario correcto.
- Servicios claros.
- FAQ minima.
- Testimonios o prueba social.
- SEO title.
- SEO description.
- Schema LocalBusiness.

- Formulario probado.
- Chatbot probado.
- Reserva probada si aplica.
- Tienda probada si aplica.
- Dashboard probado.
- Reporte inicial generado.
- Exportacion CSV verificada.
- Backend healthcheck correcto.
- Token admin configurado en produccion.
- CORS configurado en produccion.

Anexo D. Glosario

CRM: sistema de gestion de contactos, leads y clientes. **Lead:** persona interesada que deja datos de contacto. **CTA:** llamada a la accion, por ejemplo reservar o llamar. **Healthcheck:** endpoint que indica si el backend esta vivo y la base es accesible. **CORS:** configuracion que controla que dominios pueden llamar a la API desde navegador. **MVP:** producto minimo viable. **Dry-run:** simulacion o preparacion sin ejecutar envio automatico real. **Schema LocalBusiness:** datos estructurados para buscadores. **API base:** URL del backend usada por un frontend desplegado en otro dominio.